Comparison of Neural Network Implementation: From Scratch vs PyTorch

Samarpan Verma
Roll Number: 24124041

June 1, 2025

Introduction

This project focuses on implementing neural networks from scratch using NumPy and comparing the results with a PyTorch-based implementation. The primary objective was to understand the underlying mechanics of neural networks, tackle real-world data challenges, and evaluate the trade-offs between manual and library-based approaches.

Exploratory Data Analysis (EDA)

Given the real-world nature of the dataset, I began by engineering new features to improve model performance. Initially, I dropped `patientId` and `appointmentId`, assuming they were not useful. However, further analysis revealed that patient uniqueness could be leveraged for feature creation.

I extracted temporal features such as month, day, hour, and minute from the scheduled and appointment dates, and applied one-hot encoding. Later, I realized that the day of the week was more meaningful, as people are less likely to miss appointments on weekends. Thus, I retained the day-of-week feature and dropped the original schedule time.

A significant improvement came from introducing the **Gap Time** (the time difference between scheduling and appointment), which proved to be a crucial predictor. My biggest breakthrough, however, was adding the **No-show rate**, derived from patient ID uniqueness. This feature, which I initially overlooked, dramatically boosted my model's performance.

Neural Network Implementation from Scratch

For the from-scratch neural network, I followed the resources and worked through all the mathematical foundations. Most of the challenge lay in data preprocessing and feature engineering rather than the neural network implementation itself.

Initially, my model performed poorly due to severe class imbalance. To address this, I assigned different class weights to the "show" and "no-show" classes. By tuning these weights, I could trade off accuracy between the two classes, ultimately prioritizing a fair balance.

I also implemented a custom threshold of 0.65 (instead of the standard 0.5) for the positive class, which improved my results. The Adam optimizer with learning rate decay was used, starting with $l = \frac{l_0}{1+\text{decay}\times\text{iteration}}$, but I later switched to $l = l_0 \times \gamma^{\text{iteration}}$ to match the PyTorch approach.

Further tweaks included encoding the age feature (which I initially missed) and increasing the batch size to 512. I incorporated dropout layers as a precaution against overfitting, but they were not needed throughout the process.

The final neural network architecture was:

- 343 input features
- 64-neuron hidden layer
- 32-neuron hidden layer
- 2 output neurons (show, no-show)
- Softmax activation + categorical cross-entropy loss

PyTorch Implementation

Implementing the neural network in PyTorch was much easier and more streamlined compared to the from-scratch version. PyTorch's built-in modules for layers, optimizers, and loss functions significantly reduced development time and minimized the risk of bugs.

I used the same architecture and preprocessing pipeline as in the scratch implementation. The flexibility of PyTorch allowed for quick experimentation with learning rate schedules, batch sizes, and class weights.

Visualization: Metrics visualization was implemented using matplotlib and seaborn with scikit-learn's evaluation tools.

Results and Evaluation

Model Performance Comparison

Table 1: Consolidated Performance Metrics

Metric	**From-Scratch**	**PyTorch**
Accuracy	0.9139	0.9117
F1 Score (Macro)	0.8669	0.8690
F1 Score (Weighted)	0.9141	0.9136
PR-AUC	0.8973	0.8998
ROC-AUC	0.9693	0.9704

Table 2: From-Scratch Model: Overall Metrics

Metric	Value
Accuracy	0.9139
F1 Score (Macro)	0.8669
F1 Score (Weighted)	0.9141
PR-AUC	0.8973
ROC-AUC	0.9693

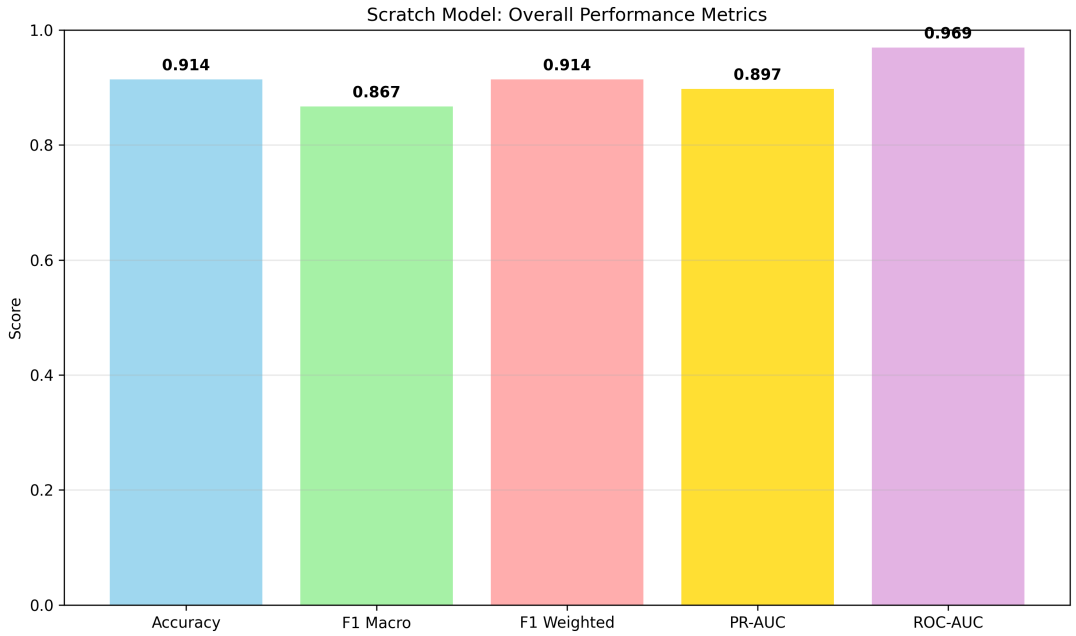

Figure 1: From-Scratch: Overall performance metrics

Conclusion

Building neural networks from scratch provided deep insight into the mechanics of forward and backward propagation, as well as the importance of feature engineering and class balancing. However, PyTorch proved to be far more efficient and user-friendly for rapid prototyping and experimentation.

Key takeaways include the critical impact of engineered features (such as gap time and no-show rate), the necessity of handling class imbalance, and the practical benefits of using a modern deep learning framework for real-world projects. PS: couldnt add preprocessed csv because it exceeded 100mb github limit

Table 3: From-Scratch Model: Per-Class Metrics

Class	F1-Score
Showed Up (0)	0.9460
No-Show (1)	0.7878

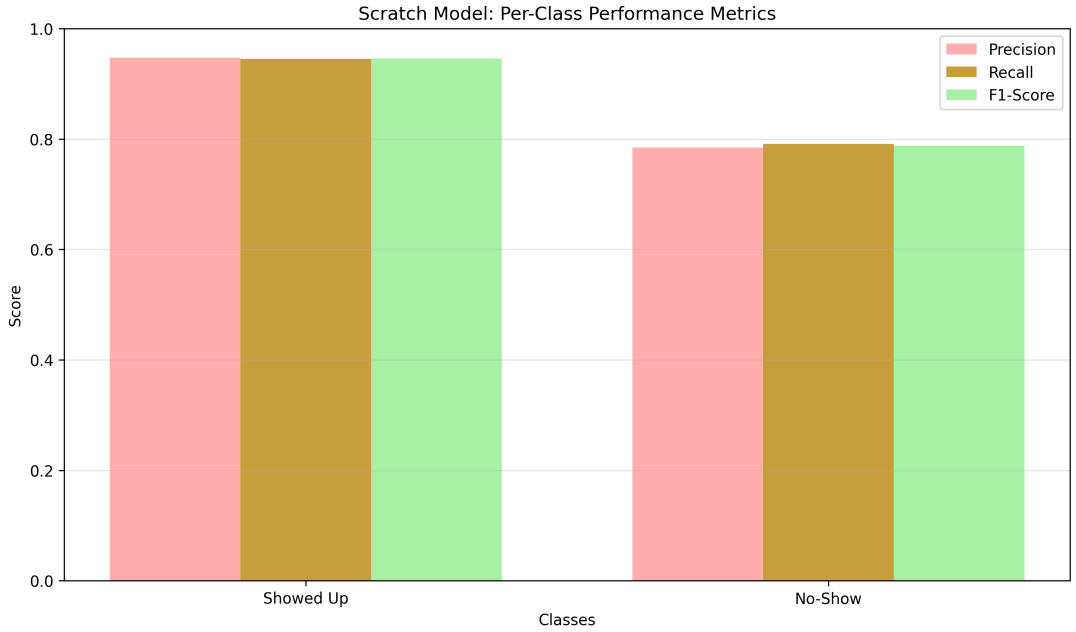

Figure 2: From-Scratch: Class-wise performance

Table 4: PyTorch Model: Overall Metrics

Metric	Value
Accuracy	0.9117
F1 Score (Macro)	0.8690
F1 Score (Weighted)	0.9125
PR-AUC	0.8998
ROC-AUC	0.9704

Table 5: PyTorch Model: Per-Class Metrics

Class	F1-Score
Showed Up	0.9438
No-Show	0.7942

Table 6: Per-Class F1-Score Comparison

Class	From-Scratch	PyTorch
Showed Up	0.9460	0.9438
No-Show	0.7878	0.7942

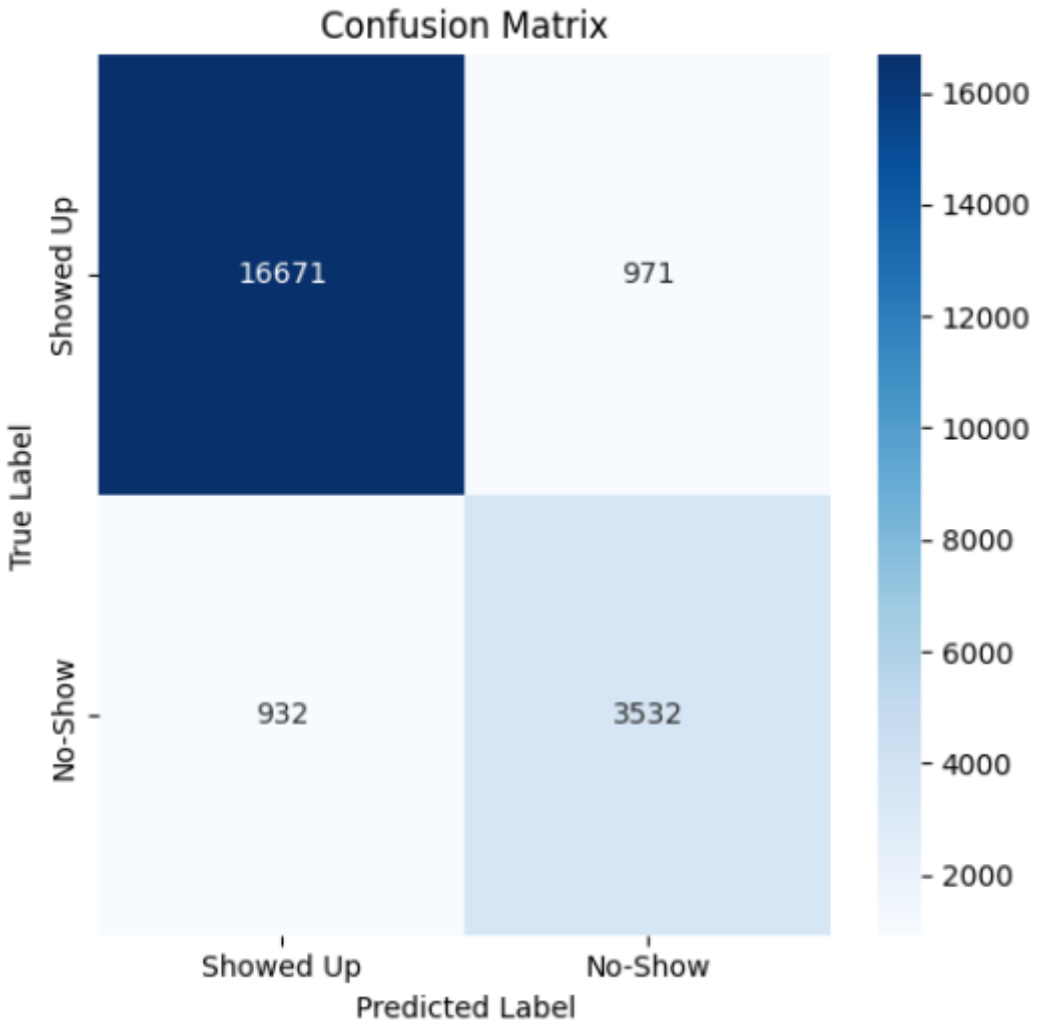

Figure 3: From-Scratch: Confusion matrix

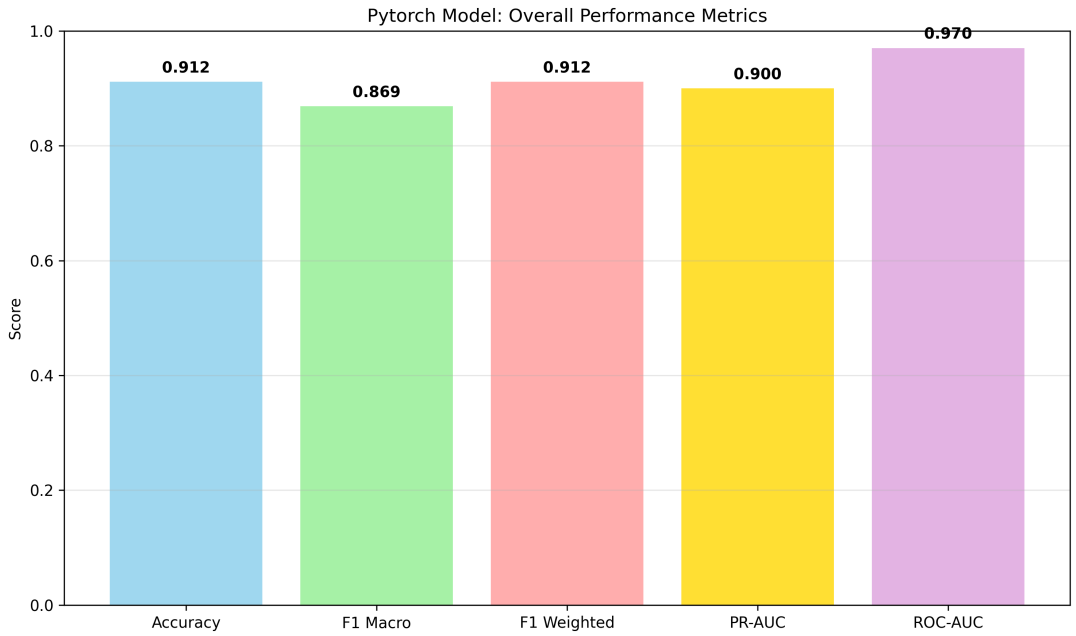

Figure 4: PyTorch: Overall performance metrics

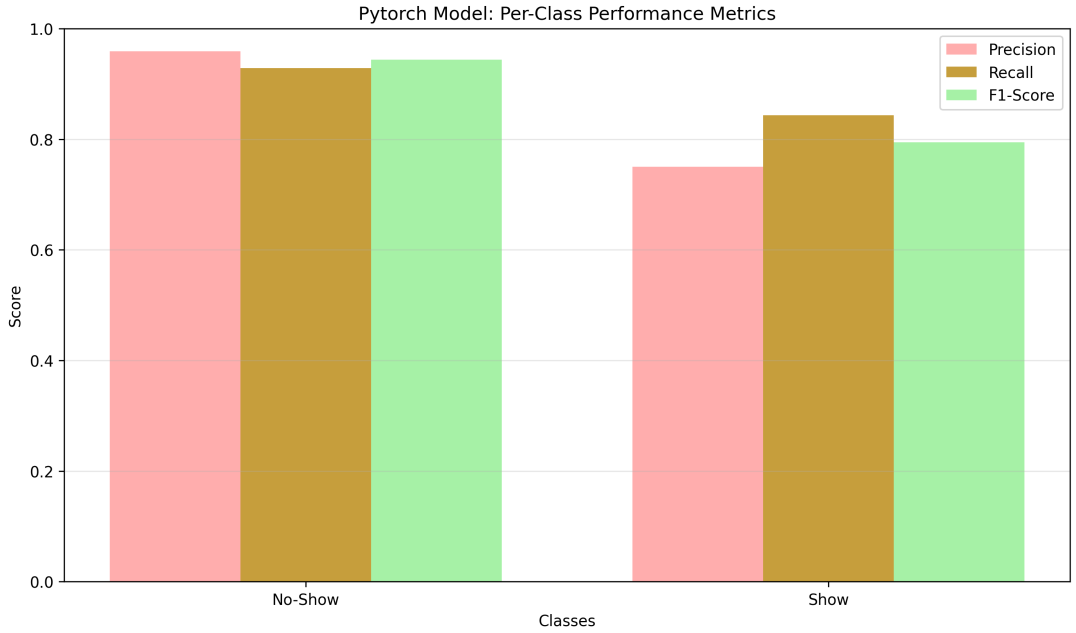

Figure 5: PyTorch: Class-wise performance

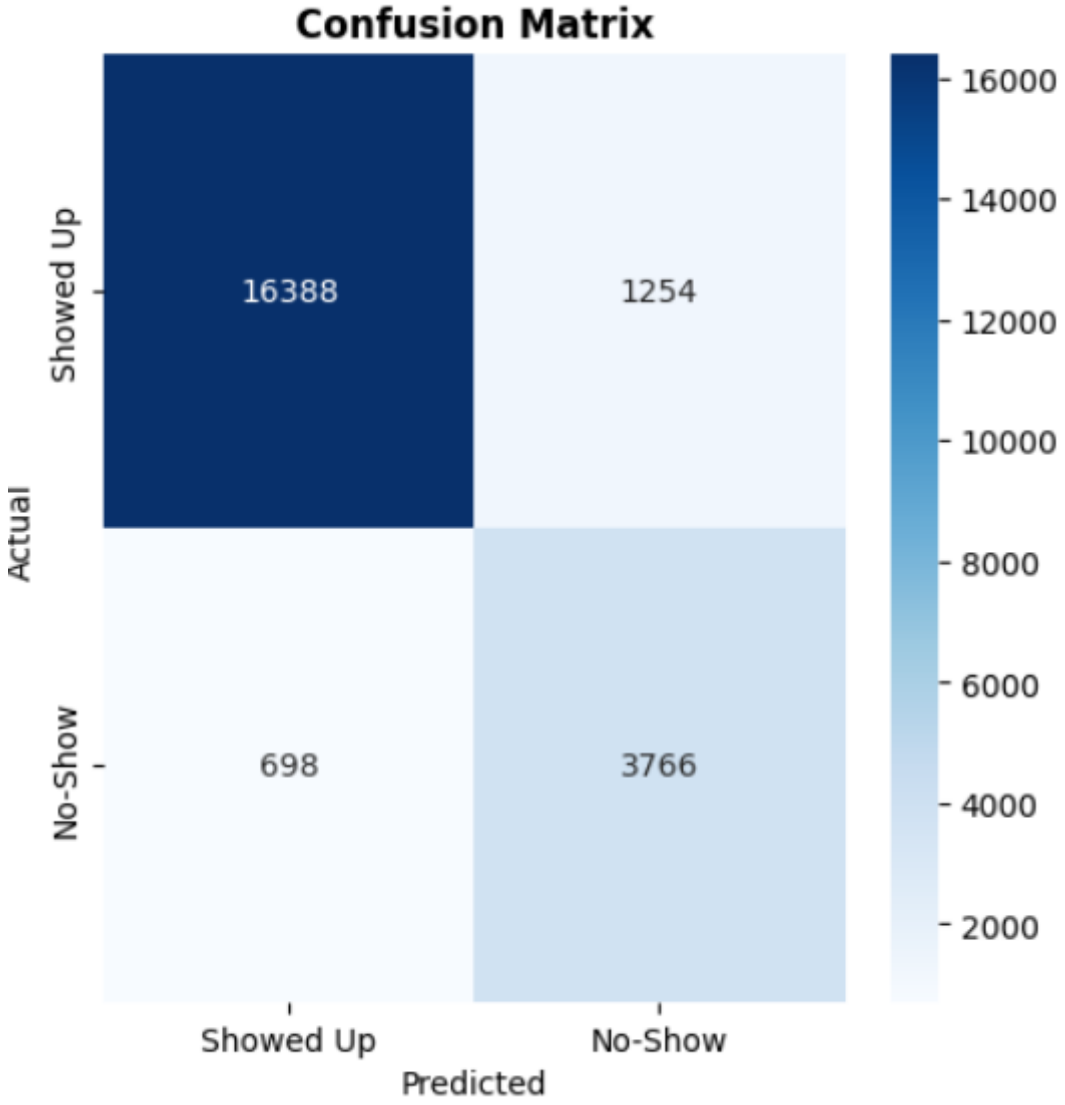

Figure 6: PyTorch: Confusion matrix